

Generative AI using LangChain

Consolidated notes — Lectures 1 to 3
Roadmap & Framing · Introduction to LangChain · LangChain Components
Course: CampusX — Generative AI using LangChain | Study notes, written up for revision

What's inside

- Part 0 — Framing:** where LangChain sits in the GenAI stack
- Part 1 — Introduction to LangChain:** the motivating problem, system design of a PDF chatbot, what LangChain is and why it exists
- Part 2 — LangChain Components:** the six building blocks (Models, Prompts, Chains, Indexes, Memory, Agents)
- Appendix A** — Glossary of terms introduced so far
- Appendix B** — Environment setup you'll need from Lecture 4 onward
- Appendix C** — Self-check questions

Part 0 — Framing: where LangChain sits

Before any code, it helps to know which problem this course is actually solving. Generative AI can be approached from two directions, and they need completely different skill sets.

Perspective	What you do	What you need to learn
Builder (model creator)	Train foundation models from scratch — GPT-class LLMs, diffusion models.	Transformer architecture, pre-training, massive compute, distributed training, RLHF.
User (application developer)	Take an existing model and build a product on top of it.	Prompt engineering, RAG, fine-tuning, agents, deployment. ← this course

Almost every job in the market right now is on the *user* side. You are not building GPT; you are building something useful with it. LangChain is the orchestration layer that makes that practical.

THE ONE-LINE SUMMARY

A foundation model is a brain in a jar. It has no memory, no access to your documents, no ability to take actions, and a raw API that differs for every provider. LangChain is the wiring that turns that brain into an application.

The layered stack



Part 1 — Introduction to LangChain

1.1 The motivating story

The course opens with a concrete product idea rather than a definition, and that framing is worth keeping: a **"chat with your PDF" application**. A student uploads a textbook and can then ask it questions in plain language — "explain chapter 3 in simple terms", "generate 5 practice questions from the linear algebra section".

The idea is old and obvious. What was missing until recently was the technology to build it. Walking through the system design shows exactly which pieces are needed — and every one of those pieces turns out to be a LangChain component.

1.2 System design of the PDF chatbot

```

[1] User uploads PDF      --> cloud storage (e.g. AWS S3)
[2] Document Loader      --> read the raw text out of the file
[3] Text Splitter        --> break it into small chunks
[4] Embedding Model      --> convert each chunk into a vector
[5] Vector Store         --> store those vectors, indexed
    ..... indexing done, one time .....
[6] User asks a question --> embed the question the same way
[7] Semantic Search      --> find the k most similar chunks
[8] Build a prompt       --> question + retrieved chunks as context
[9] LLM ("the brain")    --> reads the context, writes the answer
[10] Answer returned to user

```

Why not just use keyword search?

If the user asks "what is the sum of squares method?" and the book says "least squares regression minimises the sum of squared residuals", a keyword match may still work. But ask "how do I fit a line to noisy data?" and keyword search finds nothing — none of those words appear. The **meaning** matches; the **words** don't.

That is the case for **semantic search**: represent text as vectors that capture meaning, then measure distance between vectors. Sentences about the same idea land near each other in that space regardless of vocabulary.

Why split the document at all?

Two reasons, and both matter later:

- **Context window limits.** You cannot paste a 400-page book into a prompt. The model has a hard token limit.
- **Precision.** Embedding an entire book into one vector averages everything into mush. Small chunks produce sharp, specific vectors, so retrieval actually finds the right paragraph.

What the "brain" must be able to do

Two distinct capabilities are required of the model at step 9:

- **Natural language understanding (NLU)** — actually comprehend the question and the retrieved text.
- **Context-aware text generation** — produce an answer grounded in the supplied context, rather than from its own memory.

That second one is enforced through the system prompt, roughly: *"Answer the user's question using only the context provided below. If the answer is not in the context, say you don't know."* This one instruction is the difference between a grounded assistant and a confident liar.

RECOGNISE THIS

Steps 2–9 are exactly the **RAG** (Retrieval Augmented Generation) pattern. You've now seen the full architecture in lecture 1 — Lectures 10 to 15 just implement it piece by piece.

1.3 So what is LangChain?

LangChain is an open-source framework for building applications powered by large language models. It provides modular components plus the glue to connect them, so you assemble an application instead of writing integration code from scratch.

Notice what it is *not*: it is not a model, not a hosting provider, and not a vector database. It's the coordination layer that sits above all of those.

1.4 Why it exists — the problems it removes

Benefit	What it means in practice
Chains	Output of one step is automatically piped into the next. You describe the pipeline; you don't manually shuttle variables between calls, reformat strings, and handle each hand-off.
Model agnostic	Every provider has a different SDK, different parameter names, different response shape. LangChain puts one interface over all of them, so switching from OpenAI to Anthropic to a local Llama is a one-line change instead of a rewrite.
Complete ecosystem	Hundreds of ready-made integrations: document loaders for PDF/CSV/Notion/YouTube, text splitters, 50+ vector store connectors, retrievers, tools.
Memory & state	LLM API calls are stateless by default. LangChain provides built-in mechanisms to carry conversation history forward so a chatbot can follow a thread.
Open source & active	Free, large community, moves fast — new model providers are supported within days of release.

1.5 What you can build with it

- **Conversational chatbots** — customer support, internal helpdesk.
- **AI knowledge assistants** — question answering over private data: company docs, legal contracts, textbooks, medical records.
- **AI agents** — systems that don't just answer but *act*: book a ticket, send an email, query a database.
- **Workflow automation** — multi-step business processes driven by an LLM.
- **Summarisation and research helpers** — condense long reports, extract structured findings.

1.6 Alternatives worth knowing

Framework	Positioning
LlamaIndex	Narrower and deeper — specialises in data indexing and retrieval. Often preferred when the app is <i>purely</i> RAG over documents.
Haystack	Production-oriented NLP/search pipelines, strong on document search and evaluation tooling.

Knowing these exist matters in interviews — the follow-up question after "have you used LangChain?" is usually "why LangChain and not LlamaIndex?"

Part 2 — LangChain Components

Everything in LangChain organises into **six components**. This is the mental map for the entire rest of the course — each remaining lecture is a deep dive into one of these boxes.

Component	Purpose	Covered in
1. Models	Standard interface to any LLM or embedding model	Lecture 4
2. Prompts	Structure and templatised what you send the model	Lecture 5
3. Chains	Connect steps into a pipeline	Lectures 8-10
4. Indexes	Connect the app to external knowledge (RAG machinery)	Lectures 11-16
5. Memory	Carry conversation state across turns	Chatbot sections
6. Agents	Reasoning + tool use — models that take actions	Lectures 17-19

2.1 Models

The problem: foundation models are enormous — hundreds of GB — so you don't run them locally; you call them over an API. But every provider's API is shaped differently: different authentication, different request format, different response object. Swapping providers meant rewriting your code.

The fix: LangChain standardises the interface. Same method names, same response handling, regardless of who's behind it.

Two families of model

Type	Input → Output	Used for
Language Models	text → text	Answering, writing, reasoning, summarising
Embedding Models	text → vector (list of numbers)	Semantic search, RAG retrieval, similarity

Language models split further

	LLMs	Chat Models
Interface	string in, string out	list of messages in, message out
Roles	None	System / Human / AI
Conversation	No native support	Built for multi-turn
Status	Older, general-purpose, being phased out	Current standard

PRACTICAL RULE

Default to **Chat Models** (ChatOpenAI , ChatAnthropic , ChatGoogleGenerativeAI) for essentially everything. Plain LLM classes are legacy and appear mostly in older tutorials and blog posts.

Closed source vs open source

	Closed source (API)	Open source (weights available)
Examples	GPT (OpenAI), Claude (Anthropic), Gemini (Google)	Llama (Meta), Mistral, Gemma, DeepSeek
Access	Pay per token via API	Download weights; run locally or on your own GPU
Cost	Ongoing usage cost, no infra to manage	No per-call cost, but you pay for hardware
Control	Provider controls the model; it can change under you	Full control, fine-tunable, version-pinned
Privacy	Data leaves your machine	Data can stay entirely in-house
Run via	Provider SDK / API key	Hugging Face, Ollama, LM Studio

Parameters you'll set constantly

- `temperature` — randomness. Near **0** for factual, deterministic tasks (extraction, classification, code); **0.7-1.0+** for creative writing and brainstorming.
- `max_tokens` / `max_completion_tokens` — caps the length of the reply, which caps your cost.

2.2 Prompts

A prompt is simply the input you send the model — but it's the single highest-leverage thing you control. LLMs are **extremely sensitive** to phrasing: changing one word ("explain" → "explain like I'm five") can completely change the output. That sensitivity is the entire reason **prompt engineering** is a discipline.

Types you'll use

- **Static prompts** — hard-coded text. Fine for experiments, fragile in production because the user's input isn't controlled.
- **Dynamic / templated prompts** — a fixed skeleton with placeholders filled at runtime. This is what `PromptTemplate` gives you: reusable, validated, and it prevents the user from accidentally (or deliberately) rewriting your instructions.
- **Role-based prompts** — conversations expressed as System / Human / AI messages. The system message sets persona and rules; the rest carries the exchange.
- **Few-shot prompts** — supply worked examples of input → desired output before the real question. The most reliable way to lock down output format and tone without fine-tuning.

```
# Dynamic prompt – the shape you'll write a hundred times
template = "Summarise the research paper '{paper_name}' in a {style} tone, in {length}."
# .invoke({"paper_name": "Attention Is All You Need",
#         "style": "beginner-friendly", "length": "3 bullet points"})
```

2.3 Chains

Chains are what give the framework its name. A chain connects components into a **pipeline**, where the output of one stage is automatically the input of the next — no manual plumbing between calls.

```
Example: translate then summarise

user text --> [prompt 1 + LLM: translate to English] --> English text
--> [prompt 2 + LLM: summarise in 5 lines] --> final summary
```

Chain shapes

Type	Behaviour
Sequential	Steps run one after another, each consuming the previous output. The default shape.
Parallel	Several branches run on the same input simultaneously, then merge. E.g. generate notes <i>and</i> a quiz from one document, then combine.
Conditional	The path taken depends on a result. E.g. classify feedback sentiment → if negative, draft an apology; if positive, draft a thank-you.

2.4 Indexes

Indexes are how the application reaches **knowledge the model was never trained on** — your PDFs, your company wiki, your database. This component is the entire RAG machinery, and it's four sub-parts working in order:

Sub-component	Job
Document Loader	Pull raw content in from a source — PDF, CSV, web page, Notion, YouTube transcript, directory of files.
Text Splitter	Cut long documents into chunks small enough to embed precisely and fit a context window.
Vector Store	Store the chunk embeddings in a database built for similarity search — Chroma, FAISS, Pinecone, Qdrant, Weaviate.
Retriever	Given a query, fetch the most relevant chunks back out. This is the piece the chain actually calls at runtime.

CONNECT IT BACK

Steps 2-5 and 7 of the PDF-chatbot design in Part 1 *are* these four sub-components. Same architecture, now with names attached.

2.5 Memory

LLM API calls are stateless. Every request is independent — the model retains nothing from the previous call. Ask "Who is Sachin Tendulkar?" then "How old is he?" and the second call has no idea who "he" refers to.

ChatGPT appears to remember only because the interface resends the conversation history with every message. Memory components automate that resending — and, crucially, manage its **cost**, since history is re-sent as tokens on every single turn.

Strategy	How it works & the trade-off
Buffer memory	Send the entire transcript every time. Perfect recall; token cost grows without bound and eventually overflows the context window.
Buffer window memory	Send only the last N exchanges. Cheap and bounded; the model forgets anything older than the window.
Summarizer-based memory	Periodically compress old turns into a running summary using the LLM itself. Long conversations stay affordable; fine detail is lost in compression.
Custom memory	Persist only specific facts you care about — user's name, preferences, current order. Maximum control, most work.

2.6 Agents

The distinction that matters: **a chatbot replies; an agent acts.**

Ask a chatbot "book me a cab to the airport" and you get a helpful explanation of how one might book a cab. An agent calls the booking API and returns a confirmation number.

Two capabilities turn a chat model into an agent:

- **Reasoning** — the ability to decompose a goal into steps and decide what to do next. Usually driven by a pattern like **Chain of Thought** or **ReAct** (Reason + Act), where the model alternates between thinking and taking an action, observing the result each time.
- **Tool access** — functions it is allowed to call: web search, a calculator, a weather API, a database query, your own Python function. The model chooses which tool to use and what arguments to pass.

```
Goal: "Is my 6pm flight delayed? If yes, push my cab by an hour."

Thought --> I need the flight status.
Action  --> call flight_status_api(flight="AI-302")
Observe --> "Delayed by 75 minutes"
Thought --> It is delayed, so I should reschedule the cab.
Action  --> call cab_api.reschedule(+1 hour)
Observe --> "Confirmed for 6:15pm"
Answer  --> "Your flight is delayed 75 min; I've moved your cab to 6:15pm."
```

This loop is the destination of the whole course — Lecture 19 builds one end to end.

Appendix A — Glossary

Term	Meaning
LLM	Large Language Model — a model trained on huge text corpora to predict and generate text.
Foundation model	A large, general-purpose pre-trained model that downstream applications build on.
Token	The unit models actually process — roughly a word-piece. Pricing and context limits are measured in tokens.
Context window	Maximum tokens a model can consider in one call, prompt plus reply combined.
Embedding	A vector representation of text where geometric closeness corresponds to similarity in meaning.
Semantic search	Retrieval by meaning (via embeddings) rather than by literal keyword match.
Vector store	A database optimised for storing embeddings and running fast nearest-neighbour queries.
RAG	Retrieval Augmented Generation — retrieve relevant context first, then have the model answer using it.
Chunking	Splitting documents into smaller pieces before embedding.
Prompt engineering	Deliberately designing model input to get reliable, well-shaped output.
Few-shot prompting	Including example input/output pairs in the prompt to demonstrate the desired behaviour.
Temperature	Sampling parameter controlling randomness — low is focused and repeatable, high is varied and creative.
Stateless	Each API call is independent; the model remembers nothing between calls on its own.
Agent	An LLM equipped with reasoning and tools, able to take actions toward a goal.
Tool	A function an agent may call — API, search, calculator, database query.
ReAct	Reason + Act — the loop of thinking, acting, observing, repeating.
Hallucination	Fluent, confident output that is factually wrong. The main risk RAG is designed to reduce.

Appendix B — Setup for Lecture 4 onward

Lecture 4 (Models) is where code starts. Getting this in place now saves a stalled evening later.

```
# 1 – isolated environment
python -m venv venv
source venv/bin/activate      # Windows: venv\Scripts\activate

# 2 – core packages
pip install langchain langchain-core python-dotenv

# 3 – one or more providers
pip install langchain-openai      # OpenAI
pip install langchain-google-genai # Google Gemini (has a free tier)
pip install langchain-huggingface  # open-source models

# 4 – freeze for the repo
pip freeze > requirements.txt
```

Keep keys in a `.env` file, never in the notebook:

```
# .env <-- must be listed in .gitignore
OPENAI_API_KEY=sk-...
GOOGLE_API_KEY=...
```

```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv() # reads .env into the environment
model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
print(model.invoke("Explain LangChain in one sentence.").content)
```

BEFORE YOUR FIRST COMMIT

Add `.env`, `venv/`, `__pycache__/` and `.ipynb_checkpoints/` to `.gitignore` **now**. Once an API key is committed, deleting the file does not remove it from git history — the key must be revoked and rotated. This is the most common accident in public GenAI repos.

Appendix C — Self-check

If you can answer these from memory, lectures 1–3 are solid.

- 1 Why is keyword search insufficient for a "chat with your PDF" app, and what replaces it?
- 2 Give the two independent reasons a long document must be split into chunks.
- 3 What two capabilities must the "brain" of the system have?
- 4 Name the six LangChain components and one line on each.
- 5 What's the difference between an *LLM* and a *Chat Model* in LangChain, and which should you default to?
- 6 List three trade-offs between closed-source API models and open-source models.
- 7 Name the four sub-components of Indexes, in the order data flows through them.
- 8 Why is memory needed at all, given ChatGPT clearly "remembers" your conversation?
- 9 Buffer memory vs window memory vs summary memory — what does each cost you?
- 10 What exactly separates an agent from a chatbot?
- 11 Sketch the ReAct loop for: "find the cheapest flight to Delhi next Friday and add it to my calendar."
- 12 When would you reach for LlamaIndex instead of LangChain?

Personal study notes summarising concepts from lectures 1–3 of the CampusX *Generative AI using LangChain* course. Written in my own words for revision; not affiliated with or endorsed by CampusX. Refer to the original videos for the full explanations and code walkthroughs.